

A formalization program that can compound

Lean / Mathlib research plan | Version 1.0 | 20 September 2026

Prepared for Jett Sturges. Scope: materials science, quantum algorithms, quantum information theory, and operations research. Based on the supplied 39-item inventory, supplemented by current primary-source and repository inspection.

Recommendation

Proceed, but begin with an audited downstream Lean library and one finite-dimensional pilot. The opportunity is broader than translating textbook proofs: it includes finding the right theorem, reconciling representations, repairing dependencies, and supplying reusable mathematical interfaces.

- First, correct the gap inventory. Several advertised absences already have Mathlib declarations or substantial downstream source. A filename, README, or paper claim is not a completed proof audit.
- Use a two-state Lindblad model as the first integration benchmark. Establish an exact generator, its unique stationary density, and then its explicit CPTP dynamics and convergence. Do not present this familiar model as a new mathematical theorem.
- After the pilot, choose one extension: a finite Markov-to-Lindblad bridge, a finite CAR/Hubbard development, or a narrowly identified downstream repair. Keep operations research as a sequential alternative, not a second active program.
- Treat continuum Hohenberg-Kohn, Floquet/Bloch theory, and uniform Lieb-Robinson bounds as separate research projects with their own prerequisite audits.

What this handoff establishes

This package supplies a corrected 39-item planning inventory, theorem contracts, stage gates, source provenance, and operating instructions for Cursor with Fable 5.1 on Ubuntu 24.04. It is a research and implementation plan, not a compiled Lean release. No candidate repository was built and no transitive axiom report was executed in this session.

First action in Cursor

Read README.md, AGENTS.md, ROADMAP.md, and THEOREM_CONTRACTS.md. Run only Stage 0 and the approved finite-state API probe. Return the reproducibility evidence before starting the pilot. The included FABLE_START.md is ready to paste.

Correct the premise before choosing a flagship

Positive coverage matters more than an unsuccessful search

Inventory claim	Evidence and planning consequence
#28 Hall may not be in Mathlib4	Hall is explicitly documented in Mathlib.Combinatorics.Hall.Basic. Reuse it; remove it from the missing-theorem queue. [S03]
#10 GKSL has unresolved downstream gaps	The inspected QIClean snapshot contains gksl_iff_lindbladForm. This is a source-level candidate for reuse, subject to build, axiom, and semantic review. [S06]
#22 HSW is missing throughout Lean	Lean-QIT reports HSW and contains an operational-capacity equality endpoint. Audit the exact chain before duplicating it. [S07,S08]
#23/#24 no completed QEC or fidelity	Lean-QEC reports stabilizer-code distance certificates; Lean-QIT exposes fidelity inequalities. These cover particular subtargets, not every possible QEC theorem. [S08,S11]
#32/#36/#37 KKT, MDP, Nash absent	Optlib, MDPLib, and EconCSLib provide positive downstream candidates. Their exact assumptions and proof dependencies must be checked. [S14-S17]
Files named Holevo or DeFinetti settle #21/#25	The inspected Holevo files give dimension bounds; the inspected DeFinetti module explicitly stops short of a full theorem. Read statements, not titles. [S08,S09]

The difference from a textbook formalization

A textbook supplies a relatively fixed narrative and target statements. This program must first select statements and prove that they match useful physical or optimization questions. There is no single canonical dependency order across the four subjects. Upstream integration, downstream reuse, and new formalization are different deliverables.

Replace “missing everywhere” with “no matching audited declaration located in the recorded search scope.” Reserve priority and novelty decisions until an exact statement has been compared against current source.

Evidence levels and reproducible provenance

What was inspected, and what remains to be demonstrated

Level	Meaning in this plan
M: Mathlib documented	An exact declaration or relevant module is visible in official Mathlib documentation. Stage 0 must verify it on the selected commit.
S: source inspected	A relevant downstream statement and proof text were inspected. This does not establish successful compilation or axiom cleanliness.
R: reported candidate	A primary paper, project documentation, or repository reports coverage. The precise endpoint still needs inspection.
P: partial coverage	A narrower theorem or infrastructure exists, but does not by itself establish the listed target.
U: unresolved search	No sufficiently matching endpoint was established in this bounded review. This is not proof of absence.
V: locally verified	Reserved for a future pinned build, complete export coverage, transitive axiom check, and semantic review. No new V label is awarded here.

Four quantum snapshots inspected

Repository	Commit prefix	Lean toolchain
QuAIR/Lean-QIT	c1d59b133b56	4.30.0
LionSR/QIClean	af5430a1bb70	4.35.0-rc1
zblore/csd-lean4	3ec72510d405	4.33.0
leanprover-community/physlib	d410e856abdd	4.34.0

Full hashes, manifest Mathlib revisions, inspected paths, and source links are in SOURCES.md and provenance.json. These are observed repository snapshots, not a recommendation to combine their toolchains. Public documentation is rolling and was consulted on 20 September 2026.

Text scans for sorry or axiom are useful triage only. They can miss excluded modules, imported assumptions, altered notation, or weakened statements. The selected theorem and all of its definitions need review. [S01]

Architecture: a small downstream home

Suggested repository: formal-science | Suggested namespace: FormalScience

Create one repository depending on one pinned Mathlib revision. Keep physics-specific and application-specific results downstream. Propose small broadly reusable lemmas upstream only after they have stable statements, examples, and a human reviewer. Mathlib admission is not an automatic consequence of correctness. [S02]

Layer	Responsibility	Initial status
Foundation	Existing matrix, order, finite-sum, spectral, and finite-dimensional APIs; only missing glue belongs locally.	Select in Stage 0
Quantum / Finite	Density states, adjoints, trace identities, finite Kraus maps; adapters to the chosen dependency.	Active pilot core
OpenSystems / TwoState	Weighted jump generator, component equations, stationary state; explicit dynamics follows.	First target
Bridges / Markov	Rank-one jump generators and diagonal classical dynamics; spectral consequences only under proved hypotheses.	Next candidate
Materials / Finite	Finite CAR and Hubbard operators; discrete translation symmetry.	Later branch
Optimization	Exact LP certificates or discounted finite MDPs.	Alternative branch
Research notes	Continuum DFT, Floquet theory, quantum coding, recoverability, and long-range prerequisites.	Documentation only

Actual dependency order

Finite matrices and positivity -> states and selected Kraus lemmas -> explicit two-state generator -> stationary result -> explicit channels, semigroup, and convergence. The Markov bridge branches from generators. QFT branches from finite Fourier analysis. Entropy branches from states and functional calculus; channel data processing additionally needs channels. LP certificates have no dependence on the quantum core.

Avoid accidental framework work

Do not build a universal physics DSL, a circuit compiler, an abstract category of channels, a UI, or a multi-repository federation in the first release. Import exactly the mathematics needed by the first theorem contracts. If a trusted existing library already provides the core, prefer a thin adapter over duplicate definitions.

The first mathematical target

A two-state jump model with every convention fixed

Let E_{ij} be the 2 by 2 complex matrix unit $|i\rangle\langle j|$, with basis states 0 and 1. For a matrix V define $D[V](X) = V X V^* - (V^* V X + X V^* V)/2$. Set $H = 0$ and use nonnegative real rates a and b with $\gamma = a + b > 0$.

$$L(X) = a D[E_{10}](X) + b D[E_{01}](X)$$

Here a is the $0 \rightarrow 1$ rate and b is the $1 \rightarrow 0$ rate. This weighted presentation avoids square-root algebra at first. A later bridge identifies it with jump operators $\sqrt{a} E_{10}$ and $\sqrt{b} E_{01}$. Neither unequal rates nor two strictly positive rates are needed for stationary uniqueness.

$$\begin{aligned} L(X)_{00} &= -a X_{00} + b X_{11} \\ L(X)_{11} &= a X_{00} - b X_{11} \\ L(X)_{01} &= -(\gamma/2) X_{01} \\ L(X)_{10} &= -(\gamma/2) X_{10} \end{aligned}$$

$$\rho_* = \text{diag}(b/\gamma, a/\gamma)$$

First prove ρ_* is positive semidefinite with trace 1, then $L(\rho_*) = 0$. For every density matrix ρ , prove $L(\rho) = 0$ implies $\rho = \rho_*$. If $a, b > 0$, ρ_* is faithful. If exactly one rate is zero, uniqueness remains but faithfulness fails. If $a=b=0$, every density is stationary.

Why this is a good pilot

It exposes trace and adjoint conventions, finite-index algebra, real-to-complex coercions, positivity, kernel normalization, and meaningful boundary cases. It can be independently checked on paper. It provides a useful integration benchmark even if every underlying theorem already exists elsewhere.

What it does not establish

Stationarity is not convergence. This specific family does not prove the general GKSL characterization, generic irreducibility criteria, or a physical interpretation of measured data. The generator itself is not a quantum channel. Complete positivity is a property to prove for the time-evolution maps.

The pilot extension: exact dynamics

A separate release gate, with a direct route to complete positivity

For a density matrix ρ_0 and $t \geq 0$, define $s(t) = \exp(-\gamma t)$ and $p = b/\gamma$. Set the excited-state population and coherence by the following formulas; the remaining entries follow from trace and Hermiticity.

$$\begin{aligned}\rho_{11}(t) &= a/\gamma + (\rho_{11}(0) - a/\gamma) \exp(-\gamma t) \\ \rho_{01}(t) &= \rho_{01}(0) \exp(-\gamma t/2)\end{aligned}$$

Construct Φ_t on all matrices, not just on density states. Verify $\Phi_0 = \text{Id}$ and $\Phi_{t+u} = \Phi_t \circ \Phi_u$ for $t, u \geq 0$. Establish the differential equation $d(\Phi_t X)/dt = L(\Phi_t X)$. At zero, use a right derivative or a defined real-line extension. This connects the curve to the intended evolution.

Four-Kraus construction

With $0 \leq p \leq 1$ and $0 < s \leq 1$, use the following matrices. Multiplication by the scalar square roots is understood.

$$\begin{aligned}K_0 &= \sqrt{p} \text{diag}(1, \sqrt{s}) \\ K_1 &= \sqrt{p} \sqrt{1-s} E_{01} \\ K_2 &= \sqrt{1-p} \text{diag}(\sqrt{s}, 1) \\ K_3 &= \sqrt{1-p} \sqrt{1-s} E_{10}\end{aligned}$$

Prove $\sum_j K_j^* K_j = I$ and $\Phi_t(X) = \sum_j K_j X K_j^*$. This proves complete positivity through the finite Kraus API and trace preservation by a direct identity. It avoids taking general GKSL sufficiency as an unexamined assumption.

Convergence contract

$$\|\Phi_t(\rho) - \rho_*\|_F \leq \exp(-\gamma t/2) \|\rho - \rho_*\|_F$$

Use an explicitly defined Frobenius norm, or a verified equivalent Euclidean structure. Do not assume Mathlib's default matrix norm is the desired norm. Deduce convergence as t tends to infinity. Populations relax at γ ; coherences relax at $\gamma/2$. A diagonal Hamiltonian can be added later as an independent phase factor.

These formulas are proposed mathematical specifications, checked algebraically during planning. They are not presented as Lean code or as already machine-checked results.

Stage gates and bounded work

Advance by evidence, not by generated theorem count

Stage	Deliverable	Exit evidence
0. Reconnaissance	Pinned source inventory; one representation decision; chosen dependency route; narrow API probe.	Fresh build of the selected imports; exact candidate declarations and axioms; license/provenance record.
1. Finite core	Only the state, adjoint, trace, positivity, and Kraus lemmas required by the pilot.	Two independent uses of core lemmas; exported definitions match contracts; no duplicate upstream theorem.
2. Stationary pilot	Component equations; valid ρ_* ; stationarity; uniqueness; degenerate-rate cases.	Universal symbolic rates, not a numeric example; semantic review; complete export and axiom reports.
3. Dynamics pilot	Explicit Kraus channel, semigroup and derivative identities, quantitative convergence.	CPTP and all-time behavior proved for $t \geq 0$; rate and norm conventions independently reviewed.
4. One extension	Choose Markov bridge, CAR/Hubbard, or a verified downstream gap repair.	Statement-level novelty check; written dependency budget; reusable result beyond the pilot.
5. Release / upstream	Stable downstream release and selected foundational contributions.	Clean reproduction; understandable proof style; citations, changelog, and human review.

Planning budgets, not delivery promises

Start with a reconnaissance budget of 3-5 focused sessions and a finite-core probe of 2-4 sessions. Re-estimate after actual compilation. Each session should end with a small accepted lemma, an exact failed goal, or a documented dependency decision. No estimate here is a measured Table 5.1 success rate.

If two successive sessions expose new foundational layers instead of closing the agreed probe, narrow the scope or switch dependency route. Do not manufacture progress by assuming the desired theorem. Keep only one mathematical milestone active; independent source searches and reviews may run in parallel.

Stages 2 and 3 are separate acceptable releases. A completed stationary-state result is useful even if the dynamics proof is still in progress, provided its limitations are explicit.

How Fable should work with the mathematician

A small proof task, a compiler result, and an independent review

- Specify: write the informal theorem, all hypotheses, conventions, source location, intended use, and a counterexample to at least one tempting overgeneralization.
- Search: inspect Mathlib and the selected downstream snapshot before defining anything. Record existing declarations even when their names differ from the textbook.
- Freeze: settle the theorem statement and its definitions. The implementation agent may propose changes, but must not silently weaken the conclusion or add a premise that contains it.
- Implement: work on one small lemma or one dependency-connected group. Compile immediately; return exact diagnostics for unresolved goals. Proof sketches may live in notes; incomplete release proofs may not.
- Review: inspect the fully elaborated type, relevant instances, and definitions. Check mathematical faithfulness, nonvacuity, quantifier order, dimensional restrictions, and whether the application consumes the result.
- Validate: build every release module, check every exported declaration and its transitive axioms, and rerun the minimal downstream example. Save raw logs tied to the commit.

Trust policy for this project

The initial release allows only the ordinary foundational axioms `propext`, `Classical.choice`, and `Quot.sound`, or a subset. Reject `sorryAx`, custom theorem assumptions disguised as axioms, and native-evaluation axioms in release exports. Native trust mechanisms depend on Lean version; audit actual axiom names on the pinned toolchain. This is a project policy, not a claim that all native tactics are inherently invalid. [S01]

An umbrella import or explicit build manifest must cover every intended release file. A green lake build is insufficient if a file is not in any target. Add a deliberate failing gate fixture for missing exports or forbidden assumptions when implementing the validation harness.

Human ownership

Fable is the implementation assistant; you own the mathematical specification and review. A second agent can review a fixed statement and proof diff. Do not ask either agent to self-certify novelty or upstream acceptance. For public Mathlib work, comply with its current human-understanding and AI-disclosure rules. [S02]

Inventory 1-5: quantum algorithms

No algorithm is a mandatory predecessor of the open-systems pilot

ID / target	Assessment	Recommended treatment
1. Register and circuits	S/P: downstream state and circuit infrastructure exists. A register space, circuit syntax, and measurement semantics are different layers. [S08,S09]	Use finite index types and an explicit Hilbert structure. Reuse a register API when needed. Do not build circuit syntax for the Lindblad pilot.
2. QFT unitarity	M/P + S: Mathlib has character orthogonality and ZMod.dft; downstream qft_unitary is a direct source candidate. [S04,S05,S09]	A good alternative API pilot. Normalize by $1/\sqrt{N}$, require $N > 0$, and prove the link to the chosen basis and Fourier convention.
3. Phase estimation	S/P: downstream candidates reported and inspected at repository level. Exact probability theorem needs endpoint review. [S09]	Freeze event and rounding convention. Handle exact phases, circular distance, finite register size, and ties. The $4/\pi^2$ bound is not an unspecified global success probability.
4. Grover / amplification	S/P: downstream rotation analysis and algorithm files exist. [S09]	Separate exact rotation, iteration rounding, marked fraction, and query model. Treat zero/all marked cases and unknown solution count separately. No logical dependence on QPE.
5. Shor chain	S/P: substantial downstream development exists. "End to end" still requires statement and trust review. [S09]	Split modular arithmetic, order finding, rational reconstruction, random-base probability, and factoring. Distinguish oracle/query results from uniform circuit construction and bit complexity.

Corrected dependency picture

QFT unitarity can be formalized as a finite matrix theorem before any general circuit language. Phase estimation uses controlled powers and Fourier structure. Grover uses reflections and a two-dimensional invariant subspace, not phase estimation. Shor requires both quantum correctness and substantial classical number theory.

If the goal becomes algorithm verification, choose QFT as an alternate first pilot and postpone the open-systems track. The assumption that $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ is a necessary linear progression should be removed.

Inventory 6-12: materials and open systems

Finite models and continuum theorems need separate specifications

ID / target	Assessment and bounded next action
6. Hohenberg-Kohn I / II	U: no matching audited Lean endpoint established here. Start with an abstract variational lemma; identify potential classes, antisymmetry, domains, representability, degeneracy, and the unique-continuation bridge before a continuum claim. [S18]
7. Bloch theorem	R/P: a finite circulant Bloch seed is reported, but public proof bodies were unavailable. A finite translation-symmetric model is a tractable surrogate. [S27] Continuum periodic operators need a Floquet direct-integral formulation; extended Bloch waves need not be L2 eigenvectors. [S19]
8. Fermionic Fock / CAR	U: choose finitely many modes first. Construct finite exterior or occupation-basis operators and prove CAR. Infinite Fock completion and representations are separate targets.
9. Tight-binding / Hubbard	U: build on finite CAR or an equivalent finite matrix model. Specify graph, spin, boundary conditions, and real couplings. Prove self-adjointness and number conservation before thermodynamic claims.
10. GKSL characterization	S: QICLean contains a finite-dimensional iff theorem at the inspected commit. Audit that dependency closure before proposing a new characterization. Explicit two-state dynamics is an independent benchmark. [S06]
11. Quantum Perron-Frobenius	S/P: QICLean has substantial positive-map, irreducibility, and primitivity machinery. Classify exact endpoints. Distinguish discrete channels from continuous-time semigroups before transferring irreducibility or mixing criteria. [S06,S26]
12. Lieb-Robinson bounds	U: defer until finite tensor-local observables, commutators, graph distance, and interaction norms are stable. A useful target needs explicit constants and bounds uniform in finite volume. [S20]

Prioritization

Near term: #10 as reuse/audit and a specific-model application; then a finite classical-quantum generator bridge. Medium term: #8 -> #9 or finite #7. Long term: continuum #6/#7 and general #12. The mathematical interest of a theorem is not a reliable estimate of formalization cost.

The abstract Rayleigh-Ritz contradiction in Hohenberg-Kohn is short. Showing that its hypotheses apply to interacting continuum electrons is not. Likewise, a bounded finite matrix Hamiltonian being well-defined does not solve the domain theory of an unbounded many-body operator.

Inventory 13-20: core quantum information

Reuse a compatible finite-dimensional spine

ID / target	Assessment and next action
13. States, trace, purification	S: multiple downstream implementations. Choose one density representation and verify partial-trace ordering, tensor reindexing, and purification dimension. Reuse before porting. [S06-S10]
14. POVMs / Born rule	S: finite measurement infrastructure exists downstream. Check PSD effects sum to identity and that probabilities are real, nonnegative, and normalized. Instruments are stronger than POVM effects. [S08,S09]
15. CPTP equivalences	S: Kraus/Choi/Stinespring source candidates. Prove or reuse each direction separately, with input/output dimensions and Choi normalization explicit. The pilot needs only selected Kraus facts. [S06-S10]
16. Entropy inequalities	S/P: current quantum repositories include functional calculus and entropy development. Separate each inequality and its rank/support hypotheses; source presence is not this session's proof audit. [S08,S10,S12]
17. No-go suite	S/P: downstream candidates exist. No-cloning, no-broadcasting, and no-deleting have different premises; discarding a subsystem is allowed and is not forbidden by a no-deleting theorem. [S09]
18. Discrimination	S/P: csd-lean4 has successProbPrior_le and successProbPrior_helstromTest, providing binary Helstrom bound and attainment candidates. Unambiguous discrimination remains a separate target. [S09]
19. Protocol verification	S/P: teleportation and superdense-coding candidates exist. Require the complete classical-control and measurement semantics, with outcome corrections and all normalized inputs. [S09]
20. Entanglement criteria	S/P: QIClean has relevant criteria. PPT is not a generic equivalence to separability in arbitrary dimensions. A witness must be nonnegative on the specified separable set. [S06]

Fix the proposed QIT order

States come first. Kraus maps and tensor/partial-trace identities should precede channel data processing. Entropy can develop alongside channel infrastructure, but an entropy-first route does not eliminate channel dependencies. Avoid importing an entire coding library merely for trace preservation.

Inventory 21-26: advanced quantum information

Split bundled names into actual theorem endpoints

ID / target	Assessment	Recommended treatment
21. Holevo bound	P/U: inspected Holevo-named files establish $\chi \leq \log(\dim)$, not automatically accessible information $\leq \chi$. [S08,S09]	Search the measurement-DPI route. Define a finite ensemble and a POVM-induced classical joint law; prove $I(X:Y) \leq \chi$ for every POVM before taking a supremum.
22. HSW	S/R: Lean-QIT has the endpoint <code>classicalCapacity_eq_regularizedHolevoInformation</code> and a paper describing the coding chain. [S07,S08]	Audit operational definitions, achievability, converse, and regularization. Do not claim a new single-letter capacity theorem for arbitrary quantum channels.
23. Error correction	R/P: Lean-QEC reports stabilizer theory and certified distance results. This does not settle every Knill-Laflamme or recovery-map formulation. [S11]	Split general correctability equivalence, stabilizer algebra, CSS constructions, and code-distance certificates. Reuse the matching component.
24. Fidelity	S: Lean-QIT exposes Uhlmann and Fuchs-van de Graaf developments. [S08]	Audit support and square-root conventions. Root fidelity and squared fidelity produce different displayed inequalities.
25. Quantum de Finetti	P/U: inspected Lean-QIT symmetry module explicitly does not state the full theorem. [S08]	Choose infinite exact representation, finite trace-distance approximation, or post-selection. Specify dimension, exchangeability, marginal size, and error dependence.
26. Petz / recoverability	S/P: QICLean has Petz-map and SSA-equality candidates. Approximate Fawzi-Renner recovery is a distinct stronger quantitative task. [S06]	Separate defining a CP map, trace preservation on support, recovery of one reference state, equality-case reversibility, and quantitative recovery. [S21]

This is a later branch. The finite-state pilot does not require entropy, coding capacity, or recovery theory. Source audits of these endpoints may still be valuable contributions, especially when they expose missing support conditions or disconnected wrappers.

Inventory 27-33: optimization foundations

Several purported gaps already have downstream candidates

ID / target	Assessment and next action
27. LP duality / slackness	S/R: EconCSLib and formal LP literature provide candidates. Distinguish weak duality, strong duality under feasibility/boundedness, attainment, and strict complementarity. A certificate checker is a useful smaller target. [S14,S22]
28. Hall	M: present in Mathlib4. Example declaration: <code>Finset.all_card_le_biUnion_card_iff_exists_injective</code> . Reuse it. [S03]
29. Finite minimax	S: EconCSLib has minimax-related source. Existence, a proof-producing solver, and numerical approximation are different deliverables. Avoid treating an issue about a solver as an existence gap. [S14]
30. Max-flow min-cut	S/R: candidate Lean source was located in CLRS-Lean. Build and inspect exact capacities and graph semantics before any novelty claim. Existence of equal-value witnesses differs from termination of a flow algorithm. [S23]
31. Verified simplex	U/P: tactic oracles do not by themselves prove the simplex algorithm. An untrusted oracle may still yield a kernel-checked arithmetic proof. [S13] Start with exact certificates; termination and pivot rules are later obligations.
32. KKT	S/R: Optlib and a 2025 primary paper formalize first-order conditions. Necessity requires a suitable constraint qualification. Convex sufficiency follows from a Lagrangian bound and convexity, not an obligatory minimax detour. [S15,S16]
33. Konig / Dilworth / Menger	S/P/R: downstream Konig and Dilworth candidates and a reported finite vertex-Menger proof were located; exact trust and scope remain to be audited. Specify bipartite matching, poset chain covers, and vertex/edge path versions separately. [S24,S25]

Recommended OR entry point

If the quantum dependency audit stalls, switch to exact rational LP optimality certificates or finite discounted Bellman contraction. Both have explicit inputs, hypotheses, and observable uses. Do not restart at LP strong duality solely because the original inventory labels it the cheapest missing result.

Inventory 34-39: algorithms and stochastic OR

Theorem existence, executable algorithms, and complexity are separate

ID / target	Assessment and next action
34. Shortest paths / min-cost flow	S/P: CLRS-Lean has shortest-path candidate source. Confirm what is actually proved about the algorithm. Min-cost-flow optimality is separate; specify residual graph, potential convention, and negative cycles. [S23]
35. Matroid greedy	M/P + S/R: Mathlib supplies matroid foundations; CLRS-Lean reports a greedy optimum endpoint. Audit weight assumptions, independence-vs-basis objective, and executable sorting semantics. [S23]
36. Bellman / MDPs	P: MDPLib has discounted finite-process and Bellman-backup infrastructure, with some results marked incomplete. Full optimal-control coverage was not established here. For a local target choose finite nonempty action sets, bounded rewards, and $0 \leq \text{discount} < 1$. [S17]
37. General-sum Nash	S: EconCSLib exposes StrategicGame.exists_mixed_nash_equilibrium_finite. Check finite/nonempty assumptions and imported fixed-point results before reuse. Do not label all finite general-sum existence absent. [S14]
38. Integer programming	U/P: no unified audited endpoint established here. Split valid cuts, finite branch-and-bound search, unbounded/infeasible certificates, and total unimodularity. Each needs a separate contract.
39. M/M/1	U: no matching audited endpoint established here. First prove the geometric law is normalized and satisfies balance for arrival ≥ 0 , service > 0 , and arrival $< \text{service}$. Then prove process stationarity, nonexplosion, and convergence with their own hypotheses.

Examples of specifications that change the task

Dijkstra needs nonnegative edge weights and an explicit unreachable-vertex convention. Ford-Fulkerson with unrestricted real capacities is not a universal finite-termination argument. Matroid greedy for a basis can include negative weights; maximizing over all independent sets needs a rule for skipping unfavorable weights. Discounted Bellman contraction does not automatically cover average reward or discount equal to one.

For M/M/1, require arrival ≥ 0 , service > 0 , and $r = \text{arrival}/\text{service} < 1$. Then $\pi_n = (1-r) r^n$ is only the distribution candidate. An algebraic balance calculation is not by itself a completed theorem about the stationary law of a continuous-time process.

Materials roadmap: finite before continuum

A separate ladder for each physical regime

A. Finite fermions and lattice Hamiltonians

Choose a finite ordered set of modes. Use an exterior algebra with a compatible inner product or an occupation basis with explicit parity signs. Prove the adjoint relationship between creation and annihilation, then $\{c_i, c_j\} = 0$ and $\{c_i, c_j^*\} = \delta_{ij}$. Include the sign convention in the contract. Define a finite Hubbard Hamiltonian with conjugate hopping coefficients and real on-site interaction; prove Hermiticity and particle-number conservation.

B. Discrete periodicity before Bloch analysis

For a finite periodic lattice, prove that a translation-invariant Hamiltonian respects momentum sectors and can be block diagonalized using finite Fourier analysis. State this as a finite periodic-lattice theorem. The continuum problem needs a specified lattice, fundamental cell, boundary conditions on fibers, operator domains, and a unitary Floquet transform. Bands are fiber spectra; generalized Bloch waves need not be square-integrable on all space. [S19]

C. Hohenberg-Kohn as an explicit research project

Start with a common-domain variational comparison lemma. Separate: (i) equality of densities constrains ground-state relations; (ii) a shared ground state determines the potential modulo a constant under the required nonvanishing or unique-continuation assumptions. Establish the assumptions in a concrete continuum model before naming the result a full Hohenberg-Kohn theorem. Degenerate ground spaces need a statement designed for degeneracy. [S18]

For the variational functional, choose the density domain and distinguish pure-state from ensemble representability. Specify constrained-search infima and whether minima are attained. Do not hide existence of the universal functional, ground state, or density-potential injectivity inside a structure field and count its projection as the main theorem.

D. Lieb-Robinson only after locality infrastructure

Build local observable embeddings and prove disjoint supports commute. Fix an interaction family with bounded or decaying local terms and a graph metric. Specify the constants in a commutator estimate, their dependence on the interaction, and their independence from system size. A finite-matrix bound with constants growing arbitrarily with volume is not the intended propagation theorem. [S20]

Go/no-go rule: pursue a continuum flagship only after a domain expert has reviewed the analytic theorem contract and the Lean prerequisite search has identified a bounded first publishable milestone.

A reusable next result: the Markov bridge

A genuine connection between finite open systems and stochastic models

Let the finite state set have at least two elements. For $i \neq j$, let $q_{ij} \geq 0$ denote the jump rate $j \rightarrow i$. Set $r_j = \sum_{i \neq j} q_{ij}$. Let $Q_{ij} = q_{ij}$ off diagonal and $Q_{jj} = -r_j$; column vectors evolve by $p' = Qp$. Fix a real diagonal Hamiltonian with diagonal h_i .

$$\begin{aligned} L(X) &= -i[H, X] + \sum_{i \neq j} q_{ij} D[E_{ij}](X) \\ L(\text{diag}(p)) &= \text{diag}(Qp) \\ L(X)_{ij} &= (-i(h_i - h_j) - (r_i + r_j)/2) X_{ij} \text{ for } i \neq j \end{aligned}$$

Bridge release A: generator identities

Prove the diagonal intertwining identity, the off-diagonal formula, trace annihilation, and preservation of Hermiticity. This is algebraic and can stand alone. A classical stationary probability vector yields a quantum stationary density. If every distinct pair has $r_i + r_j > 0$, stationary off-diagonal entries vanish. Combine this fact with a separately proved classical uniqueness theorem.

Bridge release B: dynamics and convergence

Prove or reuse a valid CPTP-semigroup construction; then lift the diagonal identity to evolution and combine classical mixing with coherence decay. For a finite irreducible rate matrix on at least two states, the outgoing rates are positive, but the required classical convergence result must still be imported or proved. State the norm and any quantitative spectral-gap assumptions.

An important failed shortcut

Do not infer complete positivity of $\text{Id} + tL$ from the Lindblad form. Even simple amplitude-damping generators can have non-CP Euler steps. A classical uniformization argument cannot simply be copied to full quantum matrices. Use an audited generator theorem, a valid product/limit construction, or an explicit Kraus solution.

Why this is a better extension than a second toy example

The result packages a reusable correspondence, distinguishes populations from coherences, and provides a testbed for network and open-system models. Novelty still requires a source comparison. Its value can be integration and proof reuse even when the mathematics is classical.

No bridge theorem establishes that a fitted jump model is identifiable from application data, or that a physical quantum device is needed. Those are separate modeling and empirical questions.

Alternative first branch: exact OR certificates

Keep this as a deliberate switch or a later milestone

LP certificate contract

Choose the primal convention maximize $c^T x$ subject to $Ax \leq b$ and $x \geq 0$. The dual is minimize $b^T y$ subject to $A^T y \geq c$ and $y \geq 0$. Start over exact rationals so the checker has transparent arithmetic.

```
primal feasible x + dual feasible y => c^T x <= b^T y
feasible x,y + c^T x = b^T y => both are optimal
```

Define a certificate containing x and y , a checker over rational data, and prove checker acceptance implies feasibility and optimality. If an external solver produces the certificate, that solver need not be trusted for soundness. Checker soundness is not simplex correctness, solver completeness, or polynomial-time performance.

Only after this gate consider strong duality or certificate existence, infeasibility and unboundedness certificates, branch-and-bound tree checking, or a verified pivot algorithm. Keep numerical tolerance and rational reconstruction outside the exact checker unless their relationship is separately proved.

Discounted finite MDP contract

Alternatively, select finite states, nonempty finite available actions, normalized nonnegative transition probabilities, bounded real rewards, and $0 \leq \beta < 1$. Define the Bellman optimality operator T on real value functions with an explicit sup norm.

```
||T(V) - T(W)||_infinity <= beta ||V-W||_infinity
```

Use completeness to obtain the unique fixed point, prove finite-horizon iterates converge with an error bound, extract a maximizing stationary deterministic policy, and relate its expected discounted return to that fixed point. A fixed-point equation without policy semantics is only an analytic subresult. Audit MDPLib before recreating it. [S17]

Choosing between them

Choose certificates if you want an executable artifact with a sharply defined trust boundary. Choose discounted MDPs if you want a closer link to decision processes and the proposed Markov bridge. Neither branch requires quantum entropy, continuum PDEs, or a general solver framework.

Representation choices that prevent rework

Record these in an architecture decision before Stage 1

Choice	Default and reason
Finite spaces	Use finite index types with explicit nonemptiness where density trace 1 requires it. Use a Hilbert/Euclidean structure for vectors. A bare function type may carry the wrong default norm.
States and operators	Reuse one PSD predicate and one matrix representation. Keep density matrices as a subtype or structure only if that matches the selected ecosystem; supply coercion and extensionality lemmas early.
Tensor indexing	Use product indices initially. State basis ordering and reindexing maps. A power-of-two numerical dimension needs a verified equivalence to a qubit register before circuit claims.
Channels	Use linear maps on matrices with explicit input/output spaces. Complete positivity quantifies over ancillas or uses a proved equivalent finite-dimensional criterion; positivity alone is weaker.
Norms	Name Frobenius, operator, and trace norms explicitly. Their equivalence in finite dimensions does not make their constants interchangeable.
Entropy and fidelity	Fix log base and zero-eigenvalue conventions. Relative entropy needs a support/infinite-value policy. Fix root or squared fidelity and the factor 1/2 in trace distance.
Time and rates	Real nonnegative time, nonnegative rates, and strictly positive total rate for the pilot. Separate algebraic generator equations from differentiability and semigroup assertions.
Public API	Prefer thin adapters and broadly reusable lemmas. Avoid specialized definitions that encode the desired conclusion or force the whole repository to depend on one application.

Do not import multiple incompatible quantum libraries into one project merely because each contains a desired theorem. The inspected snapshots use four different Lean versions. Evaluate one candidate dependency at a time, or port a small permitted slice with attribution and tests.

Acceptance gates and semantic failure modes

A successful build proves the formal statement that was actually written

Risk	Required evidence
The wrong theorem is formalized	Compare the fully elaborated type to the frozen informal contract. Inspect definitions, local notation, coercions, and hidden typeclass assumptions.
A conditional result is sold as unconditional	List every premise. Construct a nontrivial example satisfying them. Flag assumed recovery, uniqueness, minimax, or coding witnesses that merely restate the desired endpoint.
A release file is never compiled	Use an authoritative export list and umbrella import or explicit module targets. The gate fails if an intended file or declaration is missing.
An imported gap contaminates a theorem	Record transitive axiom output for each public theorem. Source-level sorry scans supplement, but do not replace, that report.
Toolchain drift breaks reproduction	Pin lean-toolchain, lake-manifest.json, and dependency commits. Validate upgrades on a separate branch and repeat the release gates.
Finite or numeric results are overclaimed	State dimensions, rates, fields, and quantifiers in the release notes. Check boundary cases and distinguish tested instances from universal theorems.
Upstream integration overwhelms progress	Release downstream first. Separate correct mathematical endpoints from generalization, API cleanup, and reviewer preferences.
The agent chases breadth	One active contract. Two failed focused sessions trigger a written blocker and scope decision, not new placeholders or an unrelated theorem binge.

Definition of done for one theorem

A fixed statement with provenance; a readable proof; all intended modules built from the pinned sources; accepted transitive axioms; semantic review; a meaningful use or boundary-case demonstration; and a recorded review decision. Do not optimize for theorem count, lines of Lean, or a screenshot of a green editor.

A full release adds reproducible commands, an export manifest, dependency licenses, a changelog, source links, and the exact scope of any unverified claims. Standard kernel rechecking can be added once the basic pipeline is stable. [S01]

Contribution strategy and resourcing

A useful outcome need not be a first-ever formalization

Four legitimate outputs

- A verified reuse audit: exact statements, compatible pins, and transitive trust evidence for a downstream result that others can consume.
- An API contribution: a missing adapter or foundational lemma that eliminates repeated conversions and supports more than one theorem.
- A new formal result: a precise theorem not found in the documented search scope, with a proof and a defensible novelty comparison.
- An application theorem: a model-specific guarantee whose assumptions are connected explicitly to the model, without implying empirical validation.

How to choose the next milestone

Rank candidates by verified reuse, statement clarity, dependency depth, review availability, application value, and the chance of a maintainable release. Prefer a target with an exact endpoint and a short dependency closure over a famous theorem supported only by optimistic prose. A small independent negative result or corrected hypothesis can be valuable.

Role	Practical responsibility
You	Select the mathematical question, inspect statements and edge cases, approve scope changes, and own public claims.
Fable 5.1 in Cursor	Search the selected sources, implement small Lean lemmas, compile, report exact blockers, and maintain the evidence record.
Independent reviewer	Check statement faithfulness and the proof dependencies at each stage. A domain specialist is particularly valuable for continuum or coding-theory targets.
Lean maintainer / contributor	Advise on abstraction and destination for a potential contribution. Do not assume reviewer time or automatic acceptance.

Your mathematical and ML background supports specification and audit work, but theorem-proving speed must be measured on the chosen Lean APIs. Use CPU, memory, compiler feedback, and small task scopes first; a GPU is not a prerequisite for this program.

For Mathlib submissions, understand and justify the code personally, disclose AI assistance as required, and write public comments in your own words under the current contribution policy. Keep downstream implementation and possible upstream publication as separate gates. [S02]

The first working cycle in Cursor

Concrete tasks without an open-ended “prove the list” instruction

Cycle A: establish the environment and evidence

- Read the handoff and preserve the original inventory. Create the downstream repository, choose a provisional namespace, and record Ubuntu, editor, Lean, and Lake versions. Use the project toolchain, not an assumed global installation.
- Inspect current Mathlib and one candidate quantum dependency at exact commits. Select only the state/trace/PSD/Kraus declarations needed by the pilot. Record their source paths, license, assumptions, and local build outcomes.
- Create a finite-state probe: show a basis pure state is a valid density, show the convex mixture of the two basis states is a density, and prove one trace-preservation identity. Adapt existing results wherever possible.
- Return a Stage 0 report. If the selected dependency is incompatible or too large, compare a Mathlib-only core against a small attributed port. Do not combine all surveyed libraries.

Cycle B: stationary-state release

- Approve the representation decision and theorem contracts. Define the weighted two-state generator, prove the four entry formulas, and derive the stationary density and uniqueness for $a, b \geq 0$ with $a+b > 0$.
- Prove the one-zero-rate and both-zero-rate cases. These should expose sign mistakes, accidentally assumed faithfulness, or an overstrong uniqueness claim.
- Generate the exact export list, compile all release modules, record axioms, and request the planned mathematical audit. No dynamics theorem is required to call the stationary release complete.

Cycle C: decide the next increment

After the stationary audit passes, implement the four-Kraus dynamics and convergence contract. Then choose one extension based on what the code actually made easy. If basic matrix interfaces remain expensive, consolidate them before pursuing entropy, CAR, or a second subject.

The handoff boundary

FABLE_START.md authorizes Stage 0 and the finite-state probe, then an evidence handoff. ROADMAP.md describes the later gates. This mirrors your staged implementation-and-audit workflow while preventing the first agent run from expanding into all 39 topics.

Sources and evidence 1/3

Primary sources | Accessed 20 September 2026

References identify the evidence used for reconnaissance. Source text and a paper claim are not a local build certificate. Full commit hashes and exact declaration links are also provided in the Markdown handoff.

[S01] Lean language reference: validating a proof

Official guidance on statement meaning, transitive axioms, build coverage, and kernel rechecking. Rolling documentation; consult the pinned Lean version.

<https://lean-lang.org/doc/reference/latest/ValidatingProofs/>

[S02] Mathlib contribution guidance

Official scope, downstream-project, human-understanding, and AI-assistance policies. Checked 20 September 2026.

<https://leanprover-community.github.io/contribute/index.html>

[S03] Mathlib Hall theorem

Exact documented declaration: `Finset.all_card_le_biUnion_card_iff_exists_injective`. Positive evidence of Mathlib4 coverage.

https://leanprover-community.github.io/mathlib4_docs/Mathlib/Combinatorics/Hall/Basic.html

[S04] Mathlib finite-character orthogonality

Existing character orthogonality and linear independence. A reusable QFT ingredient, not a circuit correctness claim.

https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Fourier/FiniteAbelian/Orthogonality.html

[S05] Mathlib Fourier theory on ZMod

Definitions `ZMod.dft` and inversion theorem `ZMod.dft_dft`. Check normalization before using as a quantum unitary.

https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Fourier/ZMod.html

[S06] QIClean

Inspected snapshot `af5430a1bb70`. Finite GKSL characterization, Petz recovery, and extensive quantum operator infrastructure. Reproduction and axiom audit pending.

<https://github.com/LionSR/QIClean>

[S07] Lean-QIT primary paper

Lean-QIT: Towards a Formal Infrastructure for Quantum Information Theory (2026). Reports operational coding and capacity developments including HSW.

<https://arxiv.org/html/2607.09632v1>

[S08] Lean-QIT source

Inspected snapshot `c1d59b133b56`. HSW capacity equality and fidelity endpoints inspected. Full source permalinks are in SOURCES.md.

<https://github.com/QuAIR/Lean-QIT>

[S09] csd-lean4 source

Inspected snapshot `3ec72510d405`. QFT, phase estimation, Grover, Shor, protocols, and Helstrom candidates. Their distinct semantic scopes matter.

<https://github.com/zbllore/csd-lean4>

Sources and evidence 2/3

Primary sources | Accessed 20 September 2026

References identify the evidence used for reconnaissance. Source text and a paper claim are not a local build certificate. Full commit hashes and exact declaration links are also provided in the Markdown handoff.

[S10] physlib source

Inspected snapshot d410e856abdd. QuantumInfo includes state and entropy developments; exact theorem variants must be audited.

<https://github.com/leanprover-community/physlib>

[S11] Lean-QEC primary paper

End-to-End Formalization of Quantum Error Correction (2026). Reports stabilizer/CSS infrastructure and distance certificates; does not resolve every general correctability target.

<https://arxiv.org/abs/2605.16523>

[S12] Lean-Quantum primary development

Primary 2026 account of finite quantum operator and information theory. Repository: github.com/Hayata-Yamasaki-Group/lean-quantum. Not locally built in this review.

<https://arxiv.org/html/2607.05492v1>

[S13] Mathlib linarith architecture

Untrusted search for arithmetic certificates is distinct from kernel-checked certificate verification; it is not a verified simplex algorithm.

https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html

[S14] EconCSLib

Source candidates for LP duality, finite minimax, and mixed Nash existence. Exact source paths and declarations are listed in SOURCES.md; no local build.

<https://github.com/gametheoryinlean/EconCSLib>

[S15] Formalization of smooth optimality conditions

Li, Xu, Sun, Zhou, and Wen (2025). Primary account of KKT formalization with constraint qualifications and Farkas-based necessity.

<https://arxiv.org/abs/2503.18821>

[S16] Optlib source

Project reports LICQ/linear-CQ KKT development. Convex and Slater-related variants require separate status checks.

<https://github.com/optsuite/optlib>

[S17] MDPLib source

Finite rational-probability and discounted-process infrastructure, including Bellman backup. Some work is marked incomplete; full optimal-policy coverage not established here.

<https://github.com/formalproofs/MDPLib>

[S18] Density-potential mapping and HK hypotheses

The Structure of the Density-Potential Mapping. Part I: Standard Density-Functional Theory. Used to scope uniqueness, degeneracy, and representability obligations.

<https://arxiv.org/html/2211.16627v3>

Sources and evidence 3/3

Primary sources | Accessed 20 September 2026

References identify the evidence used for reconnaissance. Source text and a paper claim are not a local build certificate. Full commit hashes and exact declaration links are also provided in the Markdown handoff.

[S19] Periodic elliptic operators

Kuchment, An overview of periodic elliptic operators. Primary mathematical survey supporting the continuum Floquet/Bloch prerequisite distinction.

<https://arxiv.org/abs/1510.00971>

[S20] Lieb-Robinson bounds

Lieb-Robinson Bounds in Quantum Many-Body Physics. Scope and locality background. Also see arXiv:1410.8174 for unbounded on-site terms and limits.

<https://arxiv.org/abs/1004.2086>

[S21] Approximate recoverability

Fawzi and Renner, Quantum conditional mutual information and approximate Markov chains. Quantitative recovery is distinct from exact Petz-map construction.

<https://arxiv.org/abs/1410.0664>

[S22] Formal LP duality

Duality theory in linear optimization and its extensions - formally verified. Primary evidence that LP variants should be compared before reimplementation.

<https://arxiv.org/abs/2409.08119>

[S23] CLRS-Lean algorithm candidates

Published declarations for max-flow/min-cut, Dijkstra invariants, and matroid greedy. Exact page links are in SOURCES.md. No execution or complete axiom audit here.

<https://tanktechnology.github.io/CLRS-Lean/>

[S24] Konig and Dilworth source candidates

Published Formalization/DilworthTheorem.lean and KonigMatching/Duality.lean candidates. Exact target definitions and imports require review.

<https://github.com/sneed-and-feed/lean-theorems-1>

[S25] Finite vertex-Menger candidate

Provider reports a packing-or-separator statement with a Lean proof link. Candidate only; proof body and trust closure not audited here.

<https://laxarchive.org/lax-17/Lax17Proofs.Exposed.vertexMenger.html>

[S26] Irreducibility of quantum Markov semigroups

Primary mathematical discussion of irreducibility, positivity improvement, and relaxation. Continuous-time QMS criteria must not be conflated with discrete channel periodicity.

<https://arxiv.org/html/2512.11517v1>

[S27] Reported finite Bloch seed

Provider reports a finite circulant/Fourier-mode result. Public proof bodies were unavailable in this inspection; no continuum formalization follows.

https://www.physatlas.ai/pages/quantum.bloch.finite_periodic_circulant/